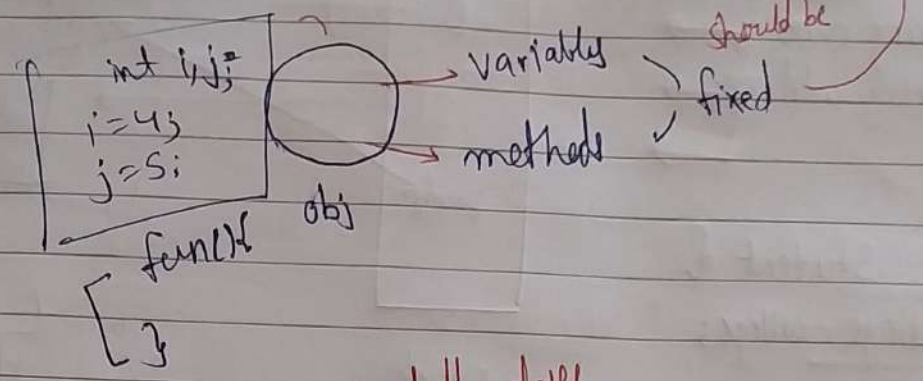
Immutable Classes



→ mutable class

```
class Student {
    int age;
    String name;
    // Constructors

    // getters
    // setters
    void markAttendance() {
      // code
    }
}
```

```
Student S1 = new Student
        (26, "Aditya");

S1.setName("Rohit");
```

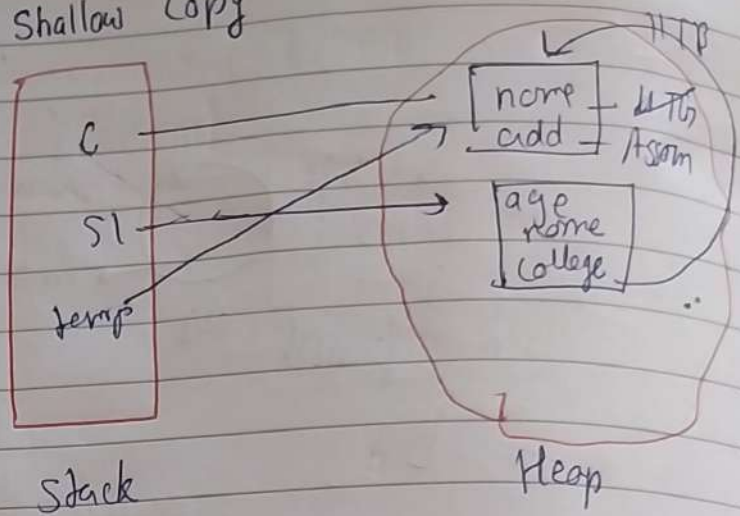
```
class CSEStudent extends Student {
    @Override
    void markAttendance()
        {} }
```

* Rules of Immutable object
  - mark my class as final
  - mark my instance variables as private & final
  - No setters
  - Defensive copy in constructor & getters

Shallow Copy

```
class College {
    String name;
    String address;
}

class Student {
    College college;
}
```

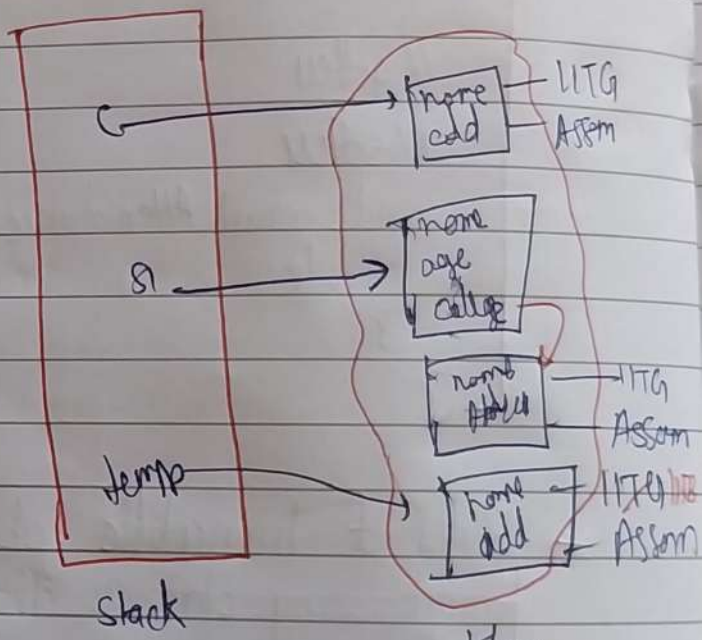



```
College c = new College("IITG", "Assam");
Student s1 = new Student(28, "Nadiya", c);
s1.getCollege().name = "IITB"
```

```
final class Student
    private final String name, age;
    private final College college;

    Student(name, age, college){
        this.college = new
            College(College name,
                college address)
    }

    public college getCollege{
        return new College(
            this.college.name, this.college.add)
    }
}
```



Deep Copy

College c = new College (IITG, Assam)
Student S1 = new Student (28, Aditya, C)
S1.getCollege().name = IITB

temp

Immutable Objects ——Imp in——→ Threads

↳ Race condition (avoided using immutable classes)